NPTEL

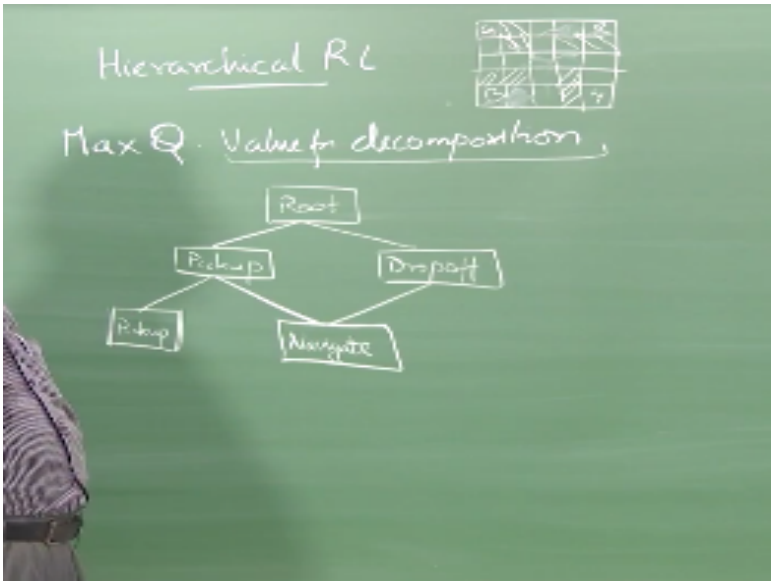
NPTEL ONLINE COURSE

REINFORCEMENT LEARNING

MAXQ

Prof. Balaraman Ravindran
Department of Computer Science and Engineering
Indian Institute of Technology Madras

(Refer Slide Time: 00:13)



Right we started looking at hierarchical reinforcement learning so we looked at several things one the need for the hierarchies SMDPs and then we looked at different kinds of optimality recursive and hierarchical and so on and so forth and then we looked at SMDP Q learning we also looked at options intra option Q learning right then HAMs and then SMDP Q learning

applied to HAMs right and then we will look at the third of the three popular hierarchical RL frame works.

And so this is called the original proposed frame work itself is called is called max Q value function decomposition right it is not called the Max Q hierarchical RL framework or anything like that the original name that Tom Dietterich gave it was Max Q value function decomposition, so when I normally teach this course right when I normally teach Max Q I leave out the value function decomposition part right.

So because that is a I mean there is value to be had in the MAX Q framework without even considering the value function decomposition, right so but I am going to cover that today, at least very briefly at least for the benefit of two students here who are trying to take advantage of the value function decomposition in their project but in general I think there is something to be said in this era of deep learning and other things to actually think about value function decomposition also right.

So I am going to talk about that and I would urge people to actually go and read more on Max Q in fact when I was a PhD student one of the things that so Andy told me was that the Max Q journal paper the journal version of the Max Q paper is like an ideal journal paper right, so this is how journal paper should be written right in for CS I mean this is how journal paper should be written and so he encouraged all of us to read that paper just for the heck of you know knowing how to write paper how to write good papers you know as a practice for that right, so I will encourage people to read the Max Q thing.

So do we have a TA here, pseudo TA yeah can you just remind them to put the Max Q journal paper to the JMLR version of it, right ask them to put it online right and yeah it is actually very well written it talks about I mean it is essentially it is almost like this like a mini PhD thesis I mean he talks about all these issues he introduces all this recursive and hierarchical optimality everything has been was introduced in that paper.

The whole concept of recursive and hierarchical optimality and then he talked about a whole set of other issues to do with hierarchies in general not just with respect to the Max Q framework of course all of us can probably at best aspire to write one set journal paper like this in our research life times so it is an ideal that you keep up there but do not try to emulate it every time because it is like he wrote one entire thesis masters' thesis this is when he was a faculty, right.

He is already a very senior faculty member at that point of time he wrote this one journal paper all by himself no students nothing involved right and if you look at it, it is almost like another PhD thesis and then he published it as one journal paper okay, so that is not a ideal strategy for people looking to graduate, so I'm just just pointing out, so even though it is a fantastic paper I encourage people to read it.

And take away take at least some good messages from it okay great so what does so we will come to the like I said we will come to the value function decomposition part in a little while so let us talk about the Max Q architecture itself right, so the basic idea here is that I'm going to decompose my problem okay into a set of sub tasks right, so I will illustrate this by the example that he used so he looked at something called the taxi domain.

So the taxi domain is like a simple 5 by 5 grid world not a very complex grid world right and then there were like G R Y and B or something like that and then there were some obstacles I am just arbitrarily putting the obstacles here, right so there were some obstacles like that you cannot walk through so if you want to go to R from here I basically have to drive like that okay. Right so there are some obstacles that kind of make that navigation hard.

And then your job is to control a taxi that moves around in this grid world right the taxi can go up down left right okay the normal grid world dynamics but apart from that the taxi can also pick up passengers and it can put down passengers right, so it can pick up a passenger only at one of these four places RGBY likewise it can put down a passenger only in one of those four places RGBY.

In other places it cannot do a pick up or put down okay, so for every move it makes it gets a - 1 right and for every passenger it delivers successfully to their intended location it gets a some reward say + 5 or something right for every move it makes it gets a - 1 right and for every passenger it delivers successfully to their intended location it gets a some reward say + 5 or something right for every move it makes it gets a - 1 for every passenger it successfully delivers it gets a + 10 so as soon as you deliver a passenger randomly another passenger will pop up in one of the other locations.

You have to go pick that passenger up and deliver and it just keeps continuing say you pick up deliver pick up deliver pick up deliver you keep doing this again and again okay so if you try to pick up a passenger from a location where there is no passenger waiting then you get a penalty let us say you get a - 2 right it is to prevent you from just keeping on randomly picking up passengers so if you try to pick up a passenger where no passenger is waiting.

You get like a - 2 and you drop off a passenger when where the passenger does not want to get down you get like a - 5 right some so the passenger gets into your cab and you know taxi in R and he says he want to go to B then you go to Y and ask the passenger to get down right well - 5 is a small penalty right, and the passenger also will not get down it is not like you can dump in throw him out at Y right.

So passenger will not get down until you go to B and drop the passenger off this is a simple thing right, you can look at all kinds of variations on this just to standardize and forestall any questions about what will happen if I try to pull the passenger off somewhere else so I am giving you all the examples right all the conditions, so this is the problem and then you just have to learn to solve this,, right so you can immediately see that there is a lot of repeated sub structure here right.

Essentially I can describe the problem to you as pick up a passenger I mean go to the place where the passenger is pick up a passenger go to the place where the passenger wants to get down drop the passenger, right so I can that is basically the policy I just have to keep repeating

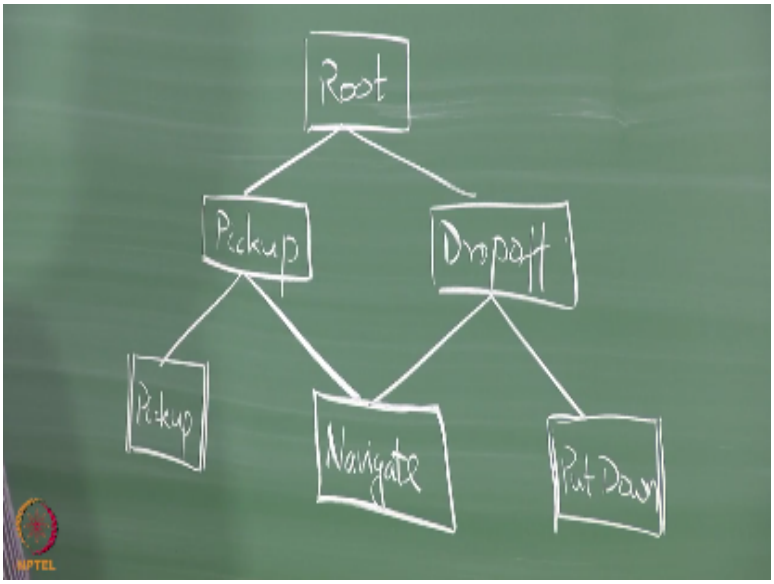
this over and over again but then I will have to do a whole bunch of things so what does it mean to say pick up a passenger and then drop off the passenger right.

So that is what I am going to decompose this problem as, so I'm going to have a root task that is going to keep doing the same task again and again so the root task is the one that I will initially execute right so the root task is going to have just two subtasks pick up the passenger drop off the passenger that is all you have to do so what are you supposed to do now, go pick up the passenger.

What I am supposed to do now, go drop off the passenger that is basically all that I have to do right, so what are the things that pick up can do pick up can well pick up okay can pick up when I put this double things let us say that it is a primitive action okay it is not going. So pick up is actually an action in the MDP right and then what else it needs to do, it needs to navigate to where the passenger is before it can pick up, correct.

So I might be here and I will say pick up the passenger from blue so essentially what should I do then, I should first go to blue and then execute the pickup action right if I try to execute the pickup action here it will fail right so likewise what about drop off, Navigate again it needs to navigate right it needs to figure out to go to the destination of the thing and then do a the opposite of pick up which is put down.

(Refer Slide Time: 10:18)



Right anything else navigate we have to implement navigate also right so navigate you need to tell navigate what are the things it can do so this is kind of like a task graph right this is like a call graph in a if you think about in a program that if you can write a call graph so there are different subroutines that you are calling right and which subroutine calls which is… then you can basically have a call graph right.

So this is like a call graph and so what is the nice thing about it is it basically gives you this structure and it is very clean to understand like in HAMs it was a little confusing I mean you could take the HAM the collection of HAMs and then draw a call graph also but here you draw the call graph itself directly so it is little easier you know interface if you will for specifying what the hierarchy should be therefore MaxQ is it is kind of more intuitive then HAMs to handle and there is a couple of one a small thing I need to point out here so navigate essentially is not a single sub task.

Right so it needs an argument I need to tell it where to navigate to right so once I decide I am going to pick up somebody from B then I will say navigate to B once I decide I am going to drop off somebody at R then I will say navigate to R so it will take an argument right so one way to think of this argument is I have 4 locations here RGBY therefore I will have 4 different sub tasks

one that says navigate to R one then navigate B navigate Y and navigate G right so I am going to have 4 different sub tasks for this.

That is one way of thinking about it in fact that is not a bad way in this particular instance because they are all different problems it is not like I can learn something from navigating to G that I can use for navigating to R may be I could but this is a small really small world right if there is a much larger world you might think of sharing knowledge across the different navigate options but now you do not have to worry about you just think of this as 4 different navigates okay.

So any questions about this set up right so this has to be necessarily a dag because if it has any cycles in it then you are doomed right so if you think about it direction of arrows is like this right so root can call pick up and drop off, pickup can call pickup and navigate drop off can call navigate and put down navigate can call north south east west after what action hmm..we will come to the structure of these things right so once the primitive action is executed you just emit the primitive action you go back this right.

So now the question is from pickup if I call navigate when do I go back to pickup or from root if I call pickup when do I go back to root right I mean I can in this particular case you can say that but how am I defining it in the structure right so that we will have to look at obviously the navigate has to go back after it has reached B suppose I call navigate B after it reaches B it has to go back right.

Or suppose I call pick up right after the passenger is in the taxi it has to go back right likewise when I call drop off after the passenger has successfully left the taxi it has to go back right I keep calling this again and again you see one thing which I want you to notice is that I am having one of those phantom phone rings have you guys felt that you feel that the phone is ringing but you do not have anything in your pocket phone is there no no apparently they have actually beginning to figure out.

That it is a real neurological condition that people are facing this phantom vibration in the like they just think that because they are so used to carrying the phones around in their pockets so they suddenly feel that that part of the thigh where the phone is in contact with they actually feel that vibratory signal this is so they are actually beginning to observe this elsewhere as well so I am not crazy I have good company so the thing that you notice here is the way I have drawn these boxes right.

Does not necessarily imply any ordering for example in the pickup task I first have to navigate only then I can pick up I cannot pickup first and then navigate right so this order is not like left to right ordering or anything right likewise in the dropoff task first I will navigate then I will put down so this is the order is not any indication of the actual sequence in which the actions will be called okay so what does this the layering imply it just says that whatever I solution I come up for the pickup task I can only use navigate and pickup as my actions right whatever solution I come for the dropoff task I can only use navigate and put down as the actions so that is all it implies okay.

So do not read anything into the order in which we are drawing the boxes okay so to specify this Max Q hierarchy so what do I do I take my original problem right and I am going to break it down into a set of subtasks right so I am going to say that I have my original problem M so M corresponds to some MDP I have my original MDP so I am going to break it down into M0 M1 after some Mk right so I am going to break it down into k sub tasks right and then for each of the subtasks right I am going to define some quantities just like we did for options.

Right for each of the subtasks I am going to define some quantities so for each sub tasks sub task Mi we are going to have what it will have it will have set of actions Ai right so how do they define exactly here so let me properly follow that notation yeah so each subtask is defined by this tuple right so where Si is the set of states in which that subtask is active right essentially these are the states in which that sub task can be running so for example for the pickup task right what you think would be the set of states in which the pickup task can be executed.

All the slow slow slow… my taxi should not have a passenger if I already have a passenger if I try to pickup another passenger that is not allowed right so the states of the taxi world should not only be the location of the taxi but also whether there is a passenger in the taxi or not in fact we can define it the other it has the location of the taxi the location of the passenger right and how many values can the location of the passenger take 5, the passenger could be in RGBY or in the taxi right if the passenger is in RGB or Y then the pickup action is valid but if the passenger is in the taxi then the pickup action is not valid right likewise

How many values can the location take of the taxi all the valid things right 25 - 8 so that many states that the taxi can be in okay so is there anything else I need to know for the state what destination of the passenger right I need to know where the passenger wants to get down so how many values that can take that can be only 4 not 5 okay so that is only 4 RGB or Y so these are the state variables location of the taxi location of the passenger and destination of the passenger okay.

And so there are bunch of things that I can execute the pickup action only if the location of the passenger and the location of the taxi is the same right or I can execute it successfully right. I can still try to execute the pickup action anywhere but it might give me a penalty right. Likewise what about navigate option, when is it valid anytime navigate is valid any time okay that is fine.

And what about put down, they are all primitive actions, so they do not really have to worry about them as subtask, but you can also define them as sub tasks right. So wherever the, now you can try to put the passenger down anywhere it is just that he will hate you right, I mean and you will get a - 10 or something right.

So you can try to put the passenger down anywhere that is fine right. Right so likewise, so you can think of Si right, what does Ai give you, Ai tells you what are the actions that are available to you right. So Ai would be a union of, would be a subset of the union of all the Ms and the primitive actions right.

So basically Ai can take from M0, M1, M2, M3 uptil Mk or N, S, E, W pickup and put down. So I have six primitive actions in my MDP so anyone of those six, any subset of those six primitive actions + any subset of any subset of M I + 1 to MIk Mk I am going to assume that I cannot call things that were numbered earlier right.

So you have to think about how will you do the ordering to enforce the dag structure right. I need a dag structure right, so I cannot have any task call M0 for example right. So I cannot have any of the sub tasks, I cannot have them call M0 as my, as an action, because M0 is a root action, that means I will be setting up a cycle.

So one way of enforcing that you do not set up a cycle is to say that the Ai cannot consider any Ms that are numbered below Ai right. This is just one way of thinking about it, you do not have to necessarily name it in that order right, you can number it in any order you want just make sure it is not a dag, it is a dag sorry, make sure it is a dag.

And here is the interesting guy, what is this Ti, Ri hat, Ri bar it is a reward associated with the sub task okay, it is a reward associated with the sub task right, it is called the pseudo reward. It is called the pseudo reward and so for example, if I want to navigate to G right, I will get a pseudo reward of something when I reach G right.

Is there anything else that I would need here, I will also need to know which of these states are my terminal states right. So where do I end the sub task right, so anything there is outside the Si will be terminal, I cannot run that sub task there right. So under the definition, so when I say navigate G it is possible only if my taxi is not at G right, because G is a terminal state right.

So one way of thinking about it is that Si gives me the active states right Ti gives me the set of terminal states or I could define Ti and whatever is not in Ti gives me the set of terminal states, I mean set of active states or I can specify Si and say whatever is not in Si is a set of terminal states and which of these are desirable terminal states that is specified by the pseudo reward right.

So for example, when I start running pickup any point of time the passenger vanishes, any point of time the passenger appears in my taxi I will end the task right. But if the passenger had appeared in my taxi only by virtue of me executing a pickup action at the location where the passenger is that is a desirable thing for you right, not from any other way of getting the passenger on the taxi right.

So when he contrived example here, but that is basically the thing. For example, we can define navigate in a slightly different way right. So navigate is active as long as I am not in RGB or Y anywhere else navigate option works fine right. So that means navigate has to terminate if I reach one of RGB or Y does it make sense right.

So navigate has to terminate when I reach one of RGB or Y, but which one of them is useful for me right, that is determined by whether it is navigate B or navigate Y or navigate R, if it is navigate R, if I terminate in R I will get a reward, pseudo reward of say +5. If I terminate in GB or Y I will get no reward right, I will just get the - 1 cost that I paid to go all the way to BY or G right, does that make sense yeah.

No do not care right, to think about no, to even if I, no there is a two things here. So even if I define navigate to end at the successful state, navigate does not know where the passenger is going right, navigate is told to go to B, so the drop off task might have made a mistake, the passenger might want to go to G, but I might tell the navigate function to go to B in which case it will anyway go to B and you cannot penalize it for going to B, because that is what it was told to do right.

So that is a different issue, so what the problem you bought up is a different issue, that the navigate is going to the wrong place right. But as long as you say navigate B and it reaches B I will give it a reward, if I say navigate B and it reaches G then I should not give it a reward right. I can keep it running I can keep it running until it reaches B that is an option I am just saying give it a negative reward, I mean give it a positive reward only when it reaches B to tell you the use of the pseudo reward function versus the terminal states right.

If you define the terminal states to be exactly those states where the reward function is 1, then there is really no difference between the two things. I am just saying that you could define multiple terminal states and then you can use the pseudo reward to distinguish between preferred terminal states and non-preferred terminal states.

No, no R I dash is only for the function, for that function Mi that is all I am solving. for that task. So it is not + 5 for the whole problem, see I have to learn the policy for navigate also right.

So for learning the policy for navigate I will use that + 5, but for solving the whole problem for the value function for dropoff or for the value function for the root I will not use the + 5. For every step it is acquiring - 1, that will be present in all the sub task, because that is the core MDP's reward that I will not touch, that will be available in all the sub tasks right.

But the - 1, the +5 sorry will be available only in the navigate, the pseudo reward will be available only in the corresponding sub task okay. So I will not be using that for the higher function, we will come to that in a minute right. So Max Q has a little bit more complexity in the learning part of it, because of all this pseudo rewards and other things.

So far, so in both the options case right and in the HAM case you are eventually, even though we are talking about hierarchies and other things you are eventually learning on a single MDP right, you are eventually learning on a single MDP, in the Max Q case you are learning on a collection of SMDPs, you are not learning on a single SMDP like in the HAM case in this.

Each one of this is actually an SMDP so M0 is an SMDP, M1 is an SMDP, M2 is an SMDP every one of this is an SMDP and what you are trying to do is collectively learn a solution for all the SMDPs in one shot right. So it makes it a little bit more complex as far as the learning goes right. And so the pseudo rewards is used for the solution of the corresponding SMDP.

So if Ri hat is there, that will be available only for solving Mi. So there will be a reward function R corresponding to the original task M and that is available for all the sub tasks. So M0 to Mk we will use R right, any Mi along the way will use only Ri hat okay make sense.

IIT Madras Production

Funded by
Department of Higher Education
Ministry of Human Resource Development
Government of India

www.nptel.ac.in